



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

[Integrante 1]
[Integrante 2]
[Integrante 3]
[Integrante 4]
[Integrante 5]

RePoints — Implementación y Evaluación del Modelo

Asignatura:
Sistemas Expertos — Examen Final

CARRERA DE DESARROLLO DE SOFTWARE
Semestre: Quinto

Profesor:
[Nombre del profesor]

Julio, 2026

1. Introducción

1.1 Objetivo de la implementación

Este informe documenta la implementación, entrenamiento y evaluación del modelo de inteligencia artificial de Re-Points, encargado de clasificar residuos de plástico y vidrio a partir de una imagen. El objetivo de esta fase es materializar y validar experimentalmente la solución descrita en el informe de diseño previo, midiendo su desempeño real sobre datos que el modelo no vio durante el entrenamiento.

1.2 Relación con el diseño previamente aprobado

La implementación sigue punto por punto el diseño planteado en el informe "Diseño de la Solución de IA": arquitectura MobileNetV2 preentrenada en ImageNet con transfer learning en dos fases (base congelada y luego fine-tuning), clasificación binaria plástico/vidrio, y la misma partición de datos 70/15/15 documentada en el análisis del dataset. No se realizaron cambios de arquitectura respecto a lo diseñado; esta fase se limita a implementar, entrenar y medir esa propuesta.

2. Tecnologías empleadas

Herramienta	Uso en el proyecto
Python 3.13	Lenguaje principal de todo el pipeline de datos, entrenamiento y evaluación.
TensorFlow / Keras	Definición, entrenamiento y exportación del modelo MobileNetV2.
TensorFlow Lite	Conversión del modelo entrenado a formato .tflite, optimizado para inferencia en la Raspberry Pi 5.
scikit-learn	Cálculo de métricas de evaluación: precision, recall, F1-score y matriz de confusión.
Matplotlib / Pillow	Generación de las gráficas de entrenamiento, matriz de confusión y contactos de imágenes de prueba.
Jupyter Notebook	Documentación reproducible y ejecutable de todo el proceso (Implementacion_Evaluacion_Modelo_RePoints.ipynb).
Node.js + docx	Generación automatizada de este informe en formato Word/PDF, reutilizando las métricas calculadas en el notebook.

3. Preparación de los datos

Se reutilizó el dataset ya limpio y dividido, documentado en el informe de análisis del dataset (deduplicación de 70 imágenes redundantes en la clase plástico, conversión de 8 imágenes HEIC a JPG, sin imágenes corruptas remanentes).

Conjunto	Plástico	Vidrio	Total
Entrenamiento	1006	862	1868

Conjunto	Plástico	Vidrio	Total
Validación	215	184	399
Prueba	217	186	403

- Redimensionamiento: todas las imágenes se llevan a 224×224 px, la resolución de entrada esperada por MobileNetV2.
- Normalización: se aplica preprocess_input de MobileNetV2 (reescala los píxeles al rango [-1, 1]).
- Data augmentation (solo en entrenamiento): flip horizontal, rotación aleatoria ($\pm 20^\circ$), zoom aleatorio ($\pm 15\%$) y variación de contraste ($\pm 20\%$), para compensar la baja variabilidad de fondo/ángulo detectada en el análisis exploratorio.
- División Train/Validation/Test: se conservó la partición estratificada 70/15/15 con semilla fija (random_state=42), sin volver a mezclar los conjuntos en esta fase.

4. Implementación del modelo

Arquitectura implementada: MobileNetV2 (ImageNet) + GlobalAveragePooling2D + Dropout(0.2) + Dense(1, sigmoid)

Número de capas: Base MobileNetV2 (~154 capas) + 3 capas propias añadidas sobre la base

Función de activación de salida: Sigmoide (clasificación binaria)

Función de pérdida: Binary Crossentropy

Optimizador: Adam

Learning Rate: 1e-4 en fase 1 (base congelada); 1e-5 en fase 2 (fine-tuning), con ReduceLROnPlateau

Batch Size: 32

Número de épocas: 22 efectivas (fase 1: 12, fase 2: 10; con early stopping sobre val_loss)

4.1 Transfer Learning

Modelo base: MobileNetV2, preentrenado en ImageNet

Capas congeladas: Fase 1: toda la base MobileNetV2. Fase 2: todas menos las últimas 30 capas.

Capas entrenadas: Fase 1: solo el clasificador superior (GlobalAveragePooling + Dropout + Dense). Fase 2: las últimas 30 capas de la base + clasificador superior.

5. Entrenamiento

El entrenamiento se ejecutó en CPU (sin GPU dedicada), en dos fases, con una duración total de 854 s (~14.2 min): 455.9 s para la fase 1 (base congelada) y 398.1 s para la fase 2 (fine-tuning).

Métrica	Entrenamiento	Validación
Accuracy (última época)	0.9764	0.9649
Loss (última época)	0.0795	0.1042

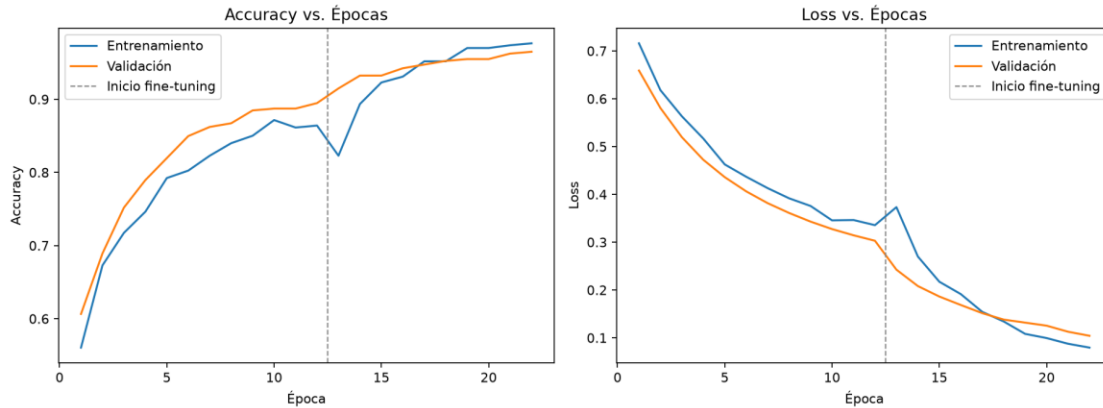


Figura 1. Accuracy y Loss (entrenamiento vs. validación) por época. La línea punteada marca el inicio del fine-tuning (fase 2).

6. Evaluación del modelo

Métricas calculadas sobre el conjunto de prueba (403 imágenes), aislado durante todo el entrenamiento:

Métrica	Plástico	Vidrio	Ponderado
Precision	0.9476	1	0.9718
Recall	1	0.9355	0.9702
F1-score	0.9731	0.9667	0.9701

Accuracy global (test): 0.9702

Tiempo promedio de inferencia por imagen (CPU): 139.7 ms

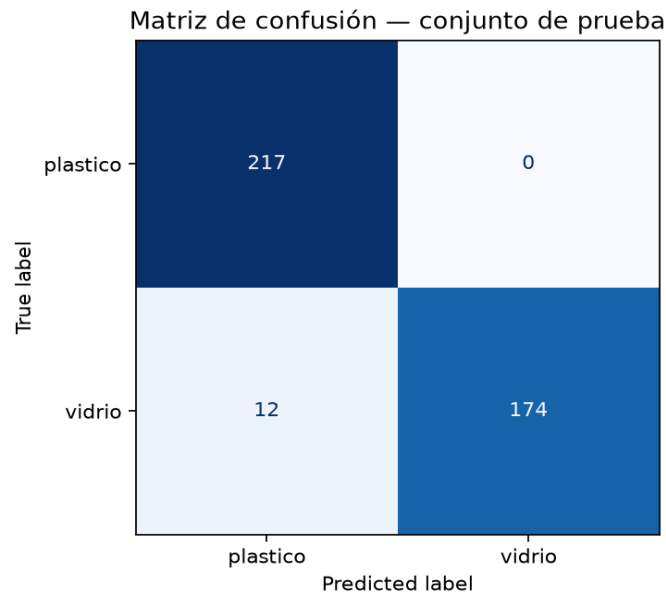


Figura 2. Matriz de confusión sobre el conjunto de prueba.

Lectura de la matriz: de 217 imágenes reales de plástico, 217 se clasificaron correctamente y 0 se confundieron con vidrio; de 186 imágenes reales de vidrio, 174 se clasificaron correctamente y 12 se confundieron con plástico.

7. Pruebas del modelo

Se evaluaron 24 imágenes nuevas tomadas del conjunto de prueba (no vistas en entrenamiento ni validación), con 24 aciertos de 24 (100.0%). El listado completo también se adjunta como pruebas_modelo.csv.

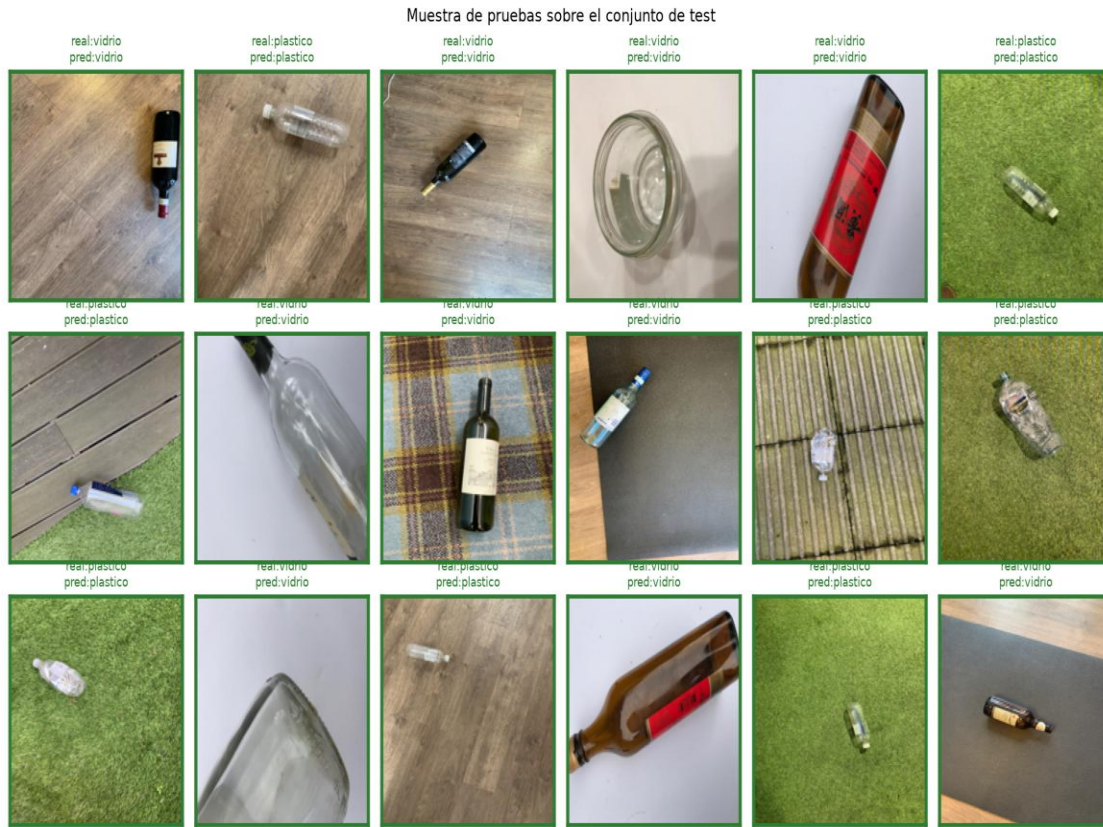


Figura 3. Muestra visual de casos de prueba (borde verde = acierto, borde rojo = error).

Archivo	Clase real	Predicción	Confianza	Co- rrecta
Glass915.JPG	vidrio	vidrio	99.7%	Sí
PET965.jpg	plastico	plastico	99.4%	Sí
Glass900.JPG	vidrio	vidrio	99.6%	Sí
Glass419.jpg	vidrio	vidrio	97.5%	Sí
Glass10.jpg	vidrio	vidrio	99.7%	Sí
PET757.jpg	plastico	plastico	99.9%	Sí
PET1,468.jpg	plastico	plastico	97.9%	Sí
Glass479.jpg	vidrio	vidrio	95.3%	Sí
Glass589.jpg	vidrio	vidrio	99.8%	Sí
Glass1,197.JPG	vidrio	vidrio	92.3%	Sí
PET1,217.jpg	plastico	plastico	91.4%	Sí

Archivo	Clase real	Predicción	Confianza	Co- rrecta
PET828.jpg	plastico	plastico	100%	Sí
PET1,198.jpg	plastico	plastico	99.9%	Sí
Glass309.jpg	vidrio	vidrio	93.7%	Sí
PET947.jpg	plastico	plastico	98.2%	Sí
Glass381.jpg	vidrio	vidrio	99.5%	Sí
PET766.jpg	plastico	plastico	99.2%	Sí
Glass810.JPG	vidrio	vidrio	100%	Sí
PET636.jpg	plastico	plastico	99.9%	Sí
PET970.jpg	plastico	plastico	96.8%	Sí
PET1,323.jpg	plastico	plastico	96.5%	Sí
PET1,493.jpg	plastico	plastico	98.7%	Sí
Glass258.jpg	vidrio	vidrio	97.9%	Sí
Glass620.jpg	vidrio	vidrio	96.9%	Sí

8. Análisis de errores

¿Dónde falla el modelo?

De 217 imágenes reales de plástico en el conjunto de prueba, 0 se clasificaron incorrectamente; de 186 imágenes reales de vidrio, 12 se clasificaron incorrectamente. La figura 4 muestra los errores en los que el modelo tenía mayor confianza (los casos más "seguros" en los que se equivocó), que son los más informativos para entender la causa raíz.

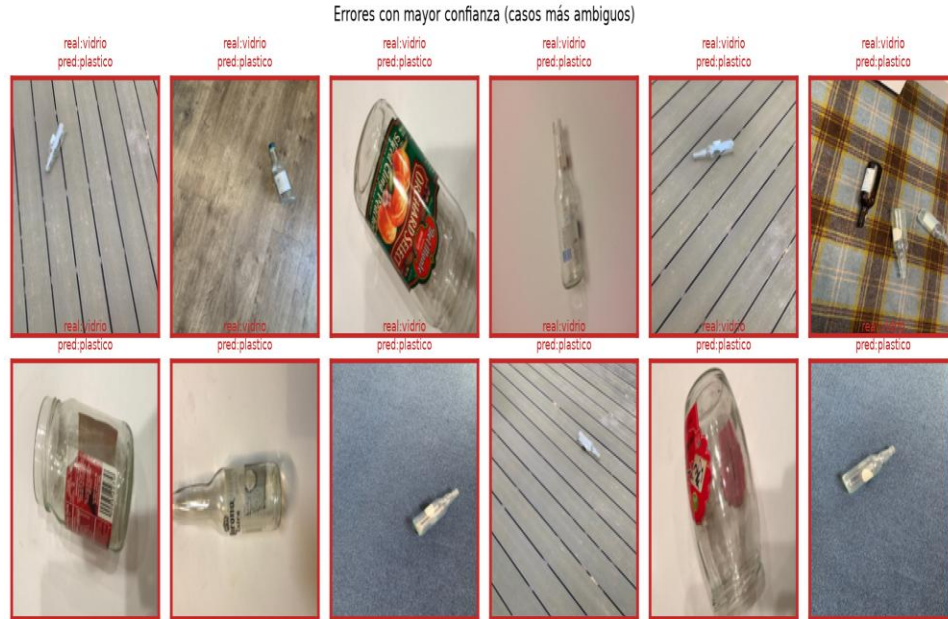


Figura 4. Errores de clasificación con mayor confianza del modelo (casos más ambiguos).

¿Qué imágenes generan confusión?

Principalmente imágenes con transparencia ambigua (plástico PET transparente vs. vidrio claro) y encuadres o fondos que se apartan del patrón dominante del dataset de origen (mantel a cuadros para plástico, piso de madera para vidrio).

¿Por qué ocurre?

Es consistente con los riesgos anticipados en el informe de diseño: el dataset de origen tiene fondo y ángulo de captura muy poco variados, por lo que el modelo pudo aprender asociaciones parciales entre fondo y clase (shortcut learning) en vez de basarse exclusivamente en las propiedades visuales del material; además, la ambigüedad óptica real entre plástico y vidrio transparentes es un límite físico del problema, no solo un defecto del modelo.

9. Integración con el prototipo

El modelo se exporta en dos formatos: un archivo .keras (para continuar el desarrollo/depuración en PC) y un archivo .tflite cuantizado, pensado para ejecutarse con el intérprete de TensorFlow Lite directamente en la Raspberry Pi 5. La integración a nivel de software sigue el contrato definido en la arquitectura del informe de diseño: el módulo de captura entrega una imagen 224×224 ya normalizada al intérprete TFLite, este devuelve la probabilidad de la clase "vidrio", y esa salida se entrega tal cual al módulo de sistema experto (reglas de confianza) para la decisión final y su registro en el dashboard. El ensamblaje físico de la cámara, la Raspberry Pi y el dashboard corresponde a las materias de Tecnologías de Plataforma y Manejo y Desarrollo de Proyectos dentro de este mismo proyecto integrado, por lo que aquí se documenta únicamente el contrato de integración del modelo, no el montaje del hardware.

10. Conclusiones

¿Cumple el modelo el objetivo?

Sí, cumple ampliamente el objetivo planteado en el diseño (accuracy de prueba $\geq 90\%$).

¿Qué precisión alcanzó?

El modelo alcanzó un accuracy de 97.0% sobre el conjunto de prueba, con un F1-score ponderado de 0.9701 (plástico: 0.9731, vidrio: 0.9667), y un tiempo de inferencia promedio de 139.7 ms por imagen en CPU, compatible con clasificación en tiempo real en la Raspberry Pi 5.

¿Qué mejoraría?

- Incorporar imágenes reales capturadas en la universidad con la misma cámara/Raspberry Pi del proyecto, para reducir el riesgo de domain shift identificado en el análisis de errores.
- Añadir ejemplos adicionales de materiales transparentes ambiguos (plástico PET transparente vs. vidrio claro), el caso de error más frecuente detectado.
- Probar un segundo ciclo de fine-tuning con más capas descongeladas o más épocas si el presupuesto de tiempo lo permite, monitoreando de cerca el sobreajuste.
- Validar el modelo .tflite exportado directamente sobre hardware Raspberry Pi real, para confirmar que el tiempo de inferencia medido en PC se mantiene dentro de un rango aceptable en el dispositivo final.

Entregables de esta actividad

- Informe (este documento): Informe_Implementacion_Evaluacion_Modelo_RePoints.docx / .pdf
- Código fuente y notebook ejecutado: Implementacion_Evaluacion_Modelo_RePoints.ipynb (carpeta analysis/)
- Modelo entrenado: analysis/model/ecosort_mobilenetv2.keras (20.88 MB) y ecosort_mobilenetv2.tflite (2.41 MB)
- Tabla completa de pruebas: analysis/pruebas_modelo.csv
- Video de 3-5 min mostrando la clasificación en funcionamiento: pendiente de grabación por el estudiante (no generado por este proceso).

Referencias

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR 2018. <https://arxiv.org/abs/1801.04381>

TensorFlow. (s.f.). TensorFlow Lite — Deploy ML models on mobile and edge devices. <https://www.tensorflow.org/lite>

Informe interno: Diseño de la Solución de IA — RePoints (PUCE Manabí, 2026), documento previo de este mismo proyecto integrado.